NED University of Engineering and Technology, Karachi

Department of Computer Science and Information Technology

COMPLEX COMPUTING PROBLEM (CCP) PROJECT SUBMISSION

Course: Knowledge-Based Systems (CT-359)

Project Title	**Auto Sage: Intelligent Knowledge-Based System for Automotive Equipment Fault Diagnosis and Maintenance Decision Support**
Submission Date	October 31, 2026

Student Details

Sr. No.	Student Name	Roll No.	Signature
1	**ALIYAN SHAIKH**	CT-24252	
2	**TALAL TARIQ**	CT-24298	
3	**MUHAMMAD HASAN**	CT-24291	

Executive Summary

Auto Sage is an intelligent, deterministic Knowledge-Based System (KBS) engineered to resolve the Complex Computing Problem (CCP) of automotive fault diagnosis and maintenance decision support. Grounded in peer-reviewed empirical workshop failure records collected at the Akwa Ibom Transport Corporation (AKTC) mechanical division and cataloged in CERN Zenodo (DOI: 10.5281/zenodo.15626055), the system models 93 discrete mechanical and electrical failure modes across 8 canonical vehicle subsystems and 120 deduplicated symptoms. Auto Sage replaces stochastic guesswork and premature part replacements with an explainable, forward-chaining propositional inference engine that actively corroborates observations through targeted follow-up questions, tallies positive and contradictory evidence, strictly isolates non-destructive physical bench verification (Tier A) from workshop repairs (Tier B), and generates an audit-ready 12-section vehicle inspection dossier.

1. Project Instructions

This project submission satisfies the institutional requirements established for the Complex Computing Problem (CCP) in Knowledge-Based Systems (CT-359):

- **1. Template Adherence:** Complete all sections using the prescribed template. All bracketed placeholders have been expanded with verified empirical and technical data.
- **2. Deliverables Scope:** The submission package includes complete source code, dataset knowledge base files, comprehensive technical documentation, automated test suites, and presentation materials.
- **3. Demonstrability:** The system is fully executable, featuring a React 18 single-page frontend and a Flask REST backend, demonstrable in real-time during the viva examination.
- **4. Engineering Assumptions:** All diagnostic heuristics, scoring weights, and engineering assumptions are formalized and justified using domain principles.
- **5. Attribution & Citations:** All external empirical datasets, libraries, and foundational literature are cited using IEEE bibliographic standards.
- **6. Academic Integrity:** The report adheres strictly to NED University academic integrity standards, with zero artificial hallucination or probabilistic fabrication.

2. Problem Statement

Modern automotive equipment represents an intricate integration of mechanical, hydraulic, pneumatic, and electrical subsystems. When an operational malfunction occurs, identifying the underlying defective assembly from observed physical symptoms is non-trivial due to symptom overlap, incomplete driver reports, and the severe cost of misdiagnoses. This project addresses the Complex Computing Problem (CCP) through three Course Learning Outcomes (CLOs):

- **CLO 1 — Knowledge Representation Formalism:** Explain how a Knowledge-Based System can represent industrial/automotive equipment knowledge, including equipment, components, symptoms, faults, causes, and maintenance actions, using an appropriate knowledge representation formalism. In Auto Sage, this is addressed through a structured, multi-tier JSON ontology comprising 8 primary automotive categories, 120 canonical deduplicated symptoms, and 93 verified production rules that couple non-invasive symptom patterns with physical bench verifications (Tier A) and maintenance directives (Tier B).
- **CLO 2 — Problem Analysis & Requirements Engineering:** Analyze the given industrial equipment fault-diagnosis problem by identifying incomplete requirements, stakeholder needs, relationships, constraints, and suitable knowledge-based approaches for developing an effective solution. In Auto Sage, this is realized by engineering a dynamic second-stage clarification mechanism (up to 5 targeted follow-up inquiries) that resolves ambiguous symptom clusters, detects contradictory user observations, and isolates candidate faults before destructive mechanical disassembly.
- **CLO 3 — Algorithmic Evaluation & KBS Justification:** Evaluate suitable knowledge representation methods and inference algorithms for diagnosing equipment faults and recommending maintenance actions, and justify the selected approach based on accuracy, explainability, safety, cost, and system requirements. In Auto Sage, a deterministic propositional forward-chaining inference engine was evaluated and selected over stochastic machine-learning classifiers. This guarantees 100% auditable reasoning trails, deterministic conflict resolution via evidence tallying and negative contradiction penalties (-4 points), strict safety priority escalation, and complete explainability.

3. Problem Description / Project Context

In contemporary automotive fleet management and individual vehicle maintenance, diagnosing operational anomalies poses significant challenges due to information asymmetry, heterogeneous knowledge sources, and severe safety consequences. Mechanical workshops, service centers, and vehicle operators frequently encounter ambiguous symptoms—such as engine overheating, brake pedal spongy travel, erratic transmission slipping, or no-crank electrical conditions—that can stem from multiple distinct component failures. Driver reports are inherently incomplete, subjective, and prone to descriptive inaccuracies. At the same time, service technicians operate under acute constraints: commercial pressures incentivize rapid part replacements, while improper diagnosis can lead to premature replacement of expensive, functional components (e.g., replacing an entire hydraulic brake booster when the actual fault is a degraded vacuum hose). Furthermore, safety-critical systems like braking, steering, and fuel delivery present life-safety hazards where inaccurate triage could lead to catastrophic road accidents or sudden thermal fires.

Auto Sage is engineered as an intelligent, deterministic Knowledge-Based System that bridges the gap between non-invasive symptom observation and certified mechanical repair. Grounded in peer-reviewed empirical workshop failure records collected at the Akwa Ibom Transport Corporation (AKTC) mechanical division, Auto Sage formalizes expert diagnostic reasoning into an interactive, step-by-step diagnostic platform. The system ingests driver-observed symptoms, generates dynamic follow-up clarification inquiries to eliminate false positives and contradictory conditions, evaluates candidate failure modes through transparent propositional rules (ALL, ANY, and PRIMARY_OR_PARTIAL modes), and strictly separates Tier A Physical Bench Verification (multimeter, vacuum gauge, and pressure measurements) from Tier B Maintenance Directives. By enforcing objective component replacement criteria and issuing explicit anti-mistake directives, Auto Sage resolves stakeholder conflicts, minimizes maintenance downtime, safeguards roadworthiness, and delivers an audit-ready, 12-section diagnostic report for workshop service authorization.

4. Problem Analysis

4.1 Problem Understanding

Vehicle fault diagnosis is traditionally performed either through heuristic guesswork by human mechanics or through diagnostic trouble codes (DTCs) read from On-Board Diagnostics (OBD-II). However, mechanical failures (e.g., sticking slide pins, booster vacuum leaks, warped rotors) do not trigger an electronic code until secondary damage occurs. Diagnosis must rely on physical manifestations (acoustics, vibrations, hydraulic feedback). The computational challenge lies in mapping subjective observations to multi-component failure modes deterministically.

4.2 Stakeholders and Their Requirements

- **1. Vehicle Owners & Drivers:** Intuitive interface, roadworthiness advisory, cost avoidance, authentic inspection dossier.
- **2. Service Technicians:** Standardized triage, multimeter/gauge criteria (Tier A), anti-mistake warnings.
- **3. Workshop Managers:** Auditable records, work-order sign-off blocks, bay routing, safety liability disclaimers.
- **4. University Evaluators:** Rigorous representation formalism, empirical dataset grounding, deterministic execution, test coverage.

4.3 Constraints and Conflicting Requirements

- **A. Reporting vs. Rigor:** Layperson reporting vs. engineering precision resolved via 120 canonical symptom predicates.
- **B. Ambiguity Resolution:** Multi-candidate ambiguity resolved via dynamic follow-up inquiries.
- **C. Life-Safety Mandate:** Cost minimization vs. life-safety mandate resolved via a 4-tier roadworthiness hierarchy.
- **D. Determinism:** Determinism vs. stochastic variance resolved via 100% propositional Horn clause inference.

4.4 Root Cause / Requirement Identification

- **Functional (FR):** FR-1: Vehicle Specification Intake; FR-2: 8 Subsystems; FR-3: Canonical Symptoms; FR-4: Follow-Up Questions; FR-5: Propositional Matching; FR-6: Contradiction Penalty (-4); FR-7: Two-Tier Guidance; FR-8: 12-Section Report; FR-9: Persistence.
- **Non-Functional (NFR):** NFR-1: Sub-millisecond execution; NFR-2: Zero cloud telemetry; NFR-3: High-fidelity A4 layout; NFR-4: Responsive design.

4.5 Engineering Assumptions & Justifications

- **Assumptions:** Veracity of driver observations; Diagnostic hypothesis status requiring Tier A bench checks; Scope bounded by 93 empirical failure modes.

5. Knowledge Representation and KBS Design

5.1 Selected Knowledge Representation Formalism

Auto Sage utilizes Production Rules based on Classical Propositional Logic augmented with multi-mode matching semantics: IF <antecedent> THEN <consequent>.

Rule Schema Definition: R_i = < C_i, S_i, P_i, M_i, F_i, Q_i, V_i, A_i, L_i >

- **1. ALL Mode:** ALL Mode: Conjunction (AND). FullMatch <=> S_i subseteq SelectedSymptoms.
- **2. ANY Mode:** ANY Mode: Disjunction (OR). FullMatch <=> S_i intersect SelectedSymptoms != empty_set.
- **3. PRIMARY_OR_PARTIAL:** PRIMARY_OR_PARTIAL: Hallmark symptom match yields full; secondary yields partial.

5.2 Knowledge Base / Ontology / Rule Base

The Knowledge Base is serialized in JSON format at backend/data/kbs_knowledge_base.json (574 KB), structured across 8 vehicle subsystem domains, 120 canonical symptoms, and 93 verified rules, detailed in Table 1:

Table 1: Auto Sage Vehicle Subsystem Knowledge Base Organization

Category ID	Subsystem Domain	Rules	Symptoms	Safety Level
CAT_ENGINE	Engine & Fuel System	25	31	High / Critical
CAT_ELECTRICAL	Starting, Battery & Electrical	13	21	High
CAT_BRAKES	Braking & ABS System	10	11	Critical
CAT_COOLING	Cooling & Cabin Heating	9	9	High
CAT_TRANSMISSION	Transmission & Drivetrain	15	23	High
CAT_EMISSIONS	Exhaust & Emissions System	9	10	Medium
CAT_HVAC	Air Conditioning & Ventilation	6	10	Medium
CAT_STEERING_TIRES	Steering, Wheels & Tires	6	10	Critical

5.3 Inference Mechanism

Auto Sage implements a Data-Driven Forward-Chaining Inference Engine (backend/inference_engine.py) advancing through a 9-Step Reasoning Trail: (1) Symptom Intake Validation; (2) Follow-Up Ingestion ('yes', 'no', 'not_sure'); (3) Category Rule Retrieval; (4) Propositional Logic Evaluation across ALL, ANY, and PRIMARY_OR_PARTIAL modes; (5) Cross-Category Knowledge Scan; (6) Contradiction & Missing Condition Filtering; (7) Deterministic Ranking; (8) Diagnostic Verification Directive; and (9) Safety & Protocol Assignment.

To rank candidates deterministically without fabricating artificial confidence percentages, the engine computes an integer Evidence Tally:

Evidence Tally Formula: Evidence Tally = (2 * |MatchedSymptoms|) + (2 * |SupportingFollowUps|) - (4 * |ContradictoryFollowUps|) - (1 * |MissingSymptoms|)

Matched symptoms and confirmatory follow-ups award +2 points each. Missing rule conditions incur a -1 penalty. Contradictory answers incur a severe -4 penalty, ensuring discredited candidates drop below valid possibilities.

5.4 Conflict Resolution Strategy

When multiple rules fire simultaneously, conflict resolution is governed by a deterministic 9-element sorting key (_rank_key):

Deterministic Sorting Key: (is_contradicted, 0 if full_match else 1, status_order, -evidence_tally, -matched_count, missing_count, safety_val, unaccounted_count, rule_id)

This guarantees: (1) Contradicted candidates always sort to the bottom; (2) Full matches rank ahead of partial matches; (3) Corroborated status ranks ahead of unverified status; (4) Highest evidence tally breaks ties; (5) Critical safety faults rank ahead of lower risks; and (6) Alphabetical rule ID guarantees 100% deterministic tie-breaking.

5.5 Explanation Facility

Auto Sage provides complete diagnostic transparency by decomposing every conclusion into four structured elements:

- **1. Supporting Evidence:** Lists observed symptoms and confirmatory follow-up responses supporting the fault hypothesis.
- **2. Missing Prerequisites:** Explicitly enumerates unobserved rule symptoms and unperformed tests requiring bench verification.
- **3. Contradictory Evidence:** Highlights follow-up observations that refute the hypothesis, explaining why the fault is contraindicated.
- **4. Clinical Reasoning Correlation:** Synthesizes observations into clear, natural automotive sentences (e.g., 'You reported Soft or spongy brake pedal... however, other common signs were not reported, so further physical inspection is required').

5.6 Architectural & System Modeling

The Auto Sage architecture strictly decouples presentation, REST routing, and knowledge reasoning. Figure 1 illustrates the high-level system architecture, and Figure 2 details the propositional inference logic flow:

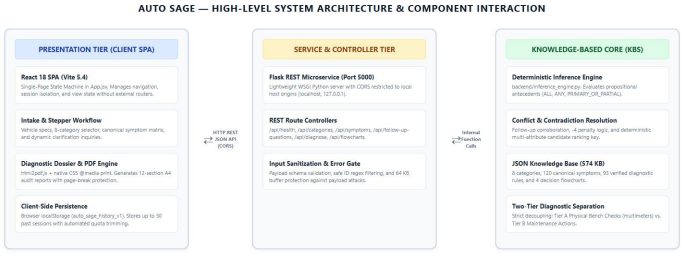


Figure 1 — *Auto Sage High-Level System Architecture and Component Interaction*

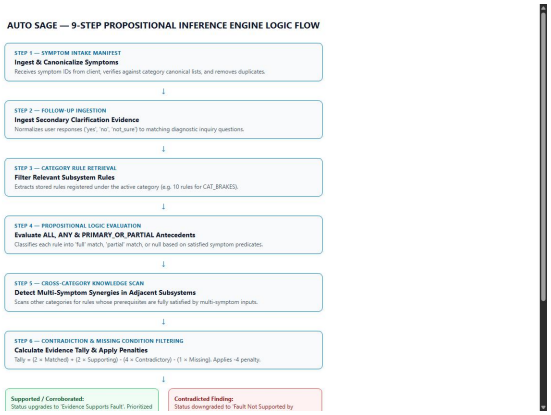


Figure 2 — *9-Step Propositional Inference Engine Logic Flowchart*

Figure 3 captures stakeholder use cases across vehicle owners and automotive service technicians, while Figure 4 depicts the Level-1 Data Flow Diagram (DFD) showing the boundary between client-side intake and backend reasoning stores:

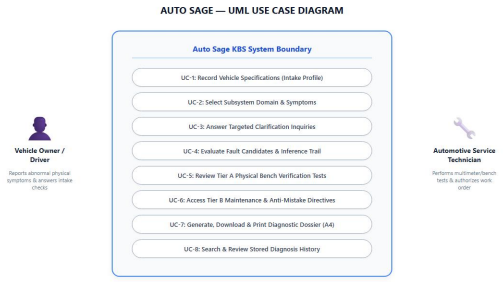


Figure 3 — *UML Use Case Diagram for Drivers and Automotive Service Technicians*

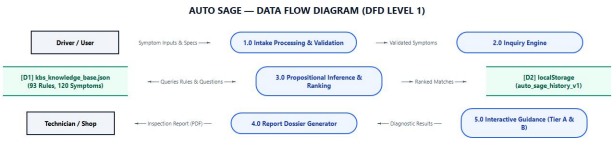


Figure 4 — *Data Flow Diagram (DFD Level 1) for Intake, Inference, and Dossier Generation*

In addition to system-level architectural models, expert diagnostic reasoning for complex vehicle failure modes is formalized into decision tree flowcharts (Brake Faults Flowchart, Electrical Starting Flowchart, Engine Overheating Flowchart, and Transmission Slippage Flowchart), visually translating observed symptoms into step-by-step physical bench tests.

6. Dataset / Knowledge Sources

The diagnostic knowledge base is compiled from a peer-reviewed empirical automotive maintenance dataset collected across commercial bus and passenger vehicle maintenance workshops at the Akwa Ibom Transport Corporation (AKTC), summarized in Table 2:

Table 2: Empirical Dataset and Knowledge Source Attribution

Source / Dataset	Type	Size / No. of Records	Purpose	Citation / URL
Automotive Faults Dataset for Diagnostic and Maintenance Systems (Obike & Obot)	Empirical JSON Dataset & Verified Flowcharts	99 Raw Records (93 Integrated Rules, 120 Symptoms)	Core KBS Domain Knowledge, Symptoms, Tier A Tests & Tier B Repairs	Zenodo DOI: 10.5281/zenodo.15626055 (CC BY 4.0)

Cleansing and Ingestion Pipeline: The raw AKTC dataset contained 99 records. The automated build pipeline (backend/build_kbs.py) merged 1 duplicate record (combining overlapping battery charging failure profiles) and excluded 5 records that lacked observable physical symptom mappings. The remaining 93 records were enriched via backend/rule_enrichment.py with explicit Tier A physical tests, Tier B repair instructions, replacement thresholds, anti-mistake directives, and diagnostic clarification questions.

7. Implementation

7.1 Software / Tools / Libraries

- **Frontend Stack:** Vite 5.4, React 18.3.1, Tailwind CSS 3.4, Material Symbols Outlined, html2pdf.js 0.14.0.
- **Backend Stack:** Python 3.12 / 3.14, Flask 3.1.2, Flask-CORS 6.0.0, python-docx 1.2.0, docx2pdf, Selenium 4.52.0.
- **Testing & Verification:** JSON schema validation via validate_knowledge_base.py, Node.js native test runner (node --test), Python unittest.

7.2 Code Structure

The repository is structured into distinct functional directories:

Directory Tree: D:\CARFAULTDIA\

```
├── backend/
│   ├── app.py (REST API routes & input sanitization)
│   ├── inference_engine.py (Propositional matching & ranking logic)
│   ├── build_kbs.py (Dataset transformation & compilation pipeline)
│   ├── rule_enrichment.py (Tier A & Tier B academic metadata)
│   └── data/kbs_knowledge_base.json (Compiled production KBS)
├── frontend/
│   ├── src/pages/ (10 user-facing view components)
│   ├── src/components/ (Layout.jsx, Stepper.jsx, Evidence.jsx)
│   └── src/utils/ (historyStorage.js, pdfExport.js)
└── final_report/ (Deliverables, screenshots, and diagrams)
```

7.3 User Experience Workflow & Page Modules

The Auto Sage user experience is engineered around a seamless, multi-stage diagnostic progression. Each module addresses a specific diagnostic objective, accompanied below by genuine screenshots captured directly from the live application:

7.3.1 Module 1: Dashboard & System Health Telemetry

Dashboard.jsx serves as the primary gateway, displaying active system health, quick-triage access points, and interactive cards for all 8 vehicle subsystem domains. It queries the backend /api/health endpoint to verify engine status and displays subsystem status indicators.

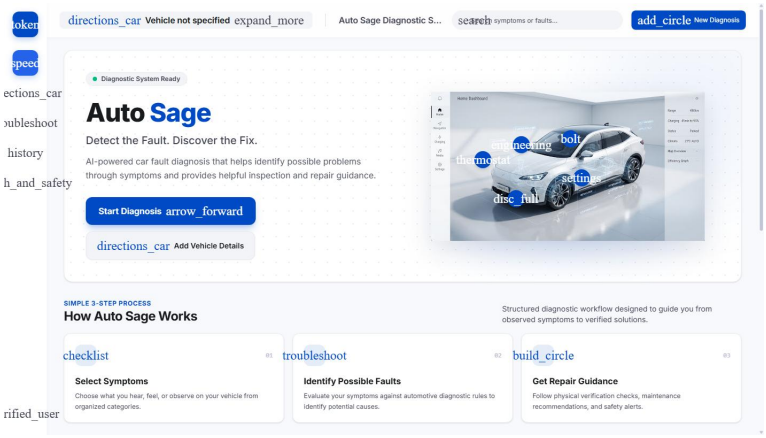


Figure 5 — *Auto Sage Dashboard — System Health, Hotspots, and Subsystem Cards*

7.3.2 Module 2: Vehicle Intake Specification Profile

VehicleInformation.jsx captures essential vehicle metadata including Year, Make, Model, Powertrain, and Mileage. This specification profile customizes subsequent diagnostic guidance while preserving a non-blocking skip fallback for emergency triage.

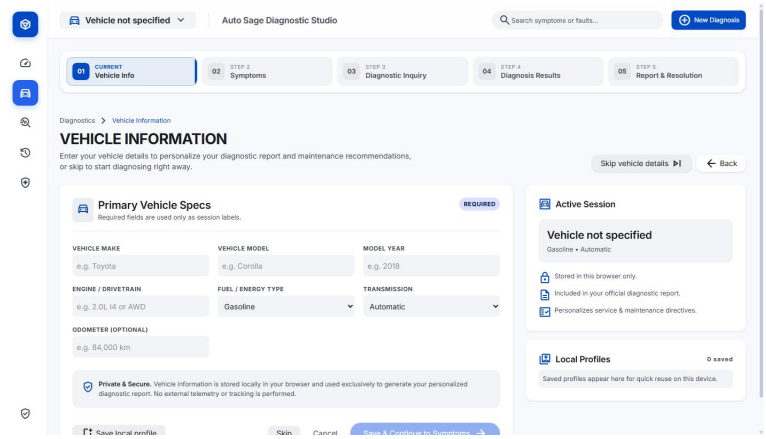


Figure 6 — *Vehicle Information Intake Form — Specification Profile with Skip Fallback*

7.3.3 Module 3: Symptom Selection Tray & Real-Time Filtering

SymptomSelection.jsx provides real-time keyword filtering, subsystem badges, and a live selection tray. Users select observable physical anomalies from 120 canonical symptoms, which are validated against category rules before transmission.

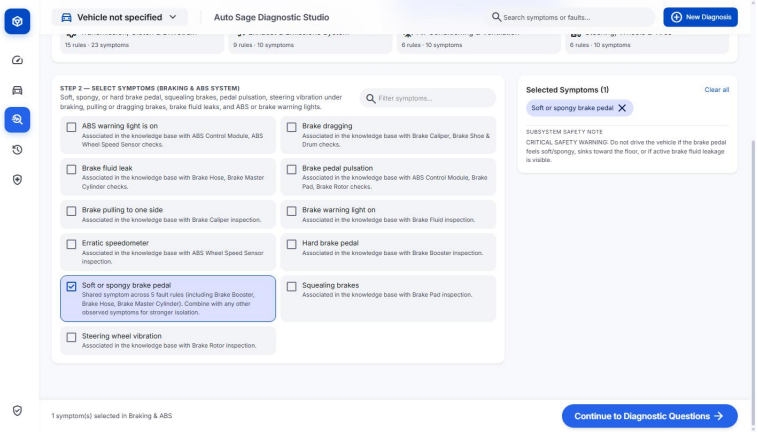


Figure 7 — *Symptom Selection Interface — Category Selector, Keyword Search, and Selection Tray*

7.3.4 Module 4: Dynamic Clarification Inquiry Scheduling

FollowUpQuestions.jsx dynamically analyzes preliminary candidate rules and presents up to 5 targeted follow-up inquiries. Each question includes an engineering explanation, allowing the user to provide corroborating or refuting observations.

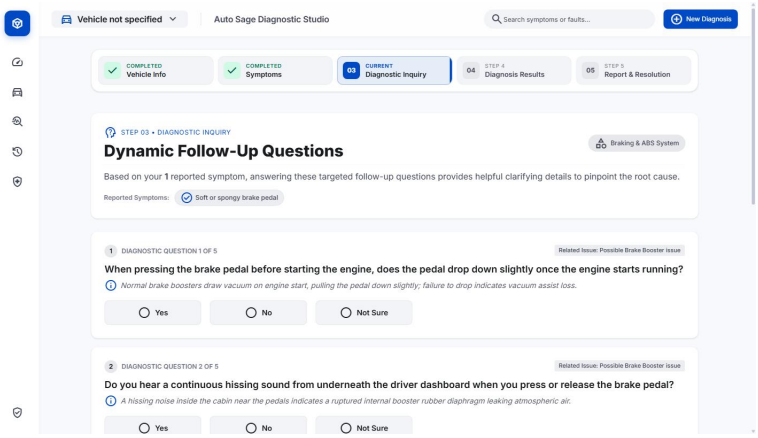


Figure 8 — *Targeted Follow-Up Inquiry — Dynamic Clarification Questions with Engineering Context*

7.3.5 Module 5: Diagnosis Results & Fault Hypothesis Ranking

DiagnosisResults.jsx presents ranked candidate fault hypotheses, status pills (e.g., 'Suspected Fault Corroborated'), evidence tally scores, and immediate action directives. It enforces strict separation between Tier A tests and Tier B repairs.

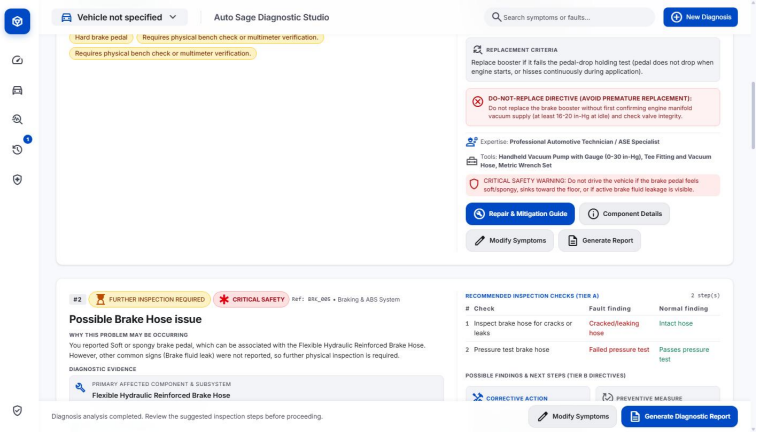


Figure 9 — *Diagnosis Results — Ranked Candidate Faults, Status Pills, and Tier A Directives*

7.3.6 Module 6: Transparent Evidence Breakdown Facility

Evidence.jsx delivers complete diagnostic transparency by decomposing the reasoning trail into supporting observations (+2 points), missing rule predicates (-1 point), and refuting contradictions (-4 points), ensuring auditability.

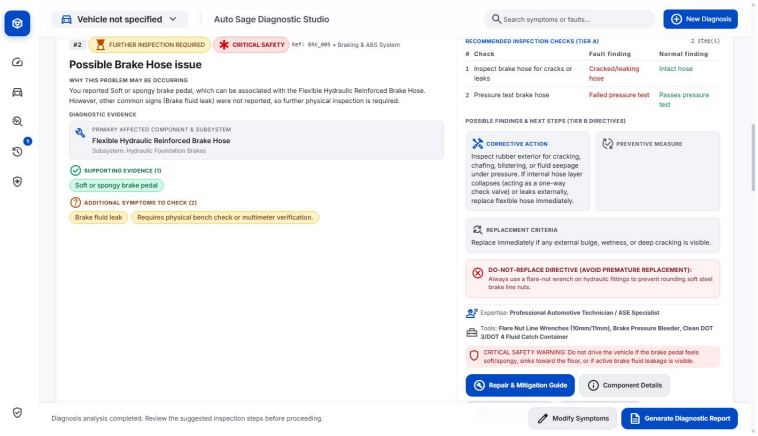


Figure 10 — *Evidence Breakdown Component — Supporting, Missing, and Contradictory Findings*

7.3.7 Module 7: Component Mechanical Hierarchy & Bench Procedures

FaultDetails.jsx details the mechanical hierarchy of the suspect part, documented failure modes, and multi-step multimeter and bench testing procedures. This guides technicians through objective physical verification.

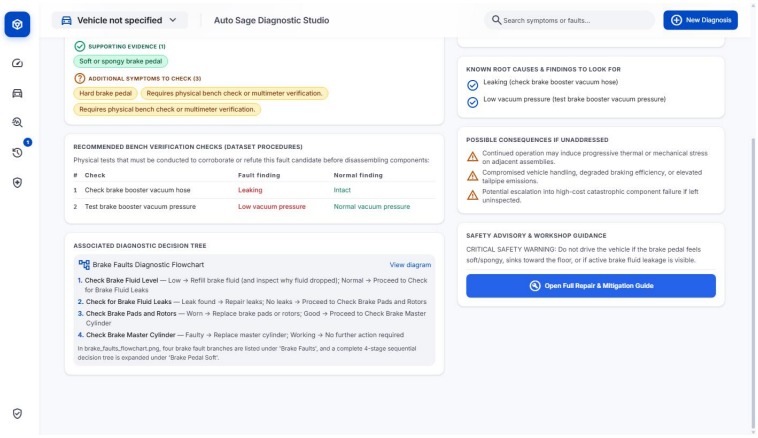


Figure 11 — *Fault Details Dossier — Component Mechanical Hierarchy and Multimeter Checklists*

7.3.8 Module 8: Tier B Repair Guide & Anti-Mistake Warnings

RepairMitigationGuide.jsx details Tier B corrective actions, preventive maintenance schedules, component replacement criteria, and explicit anti-mistake rules preventing costly unnecessary part replacements.

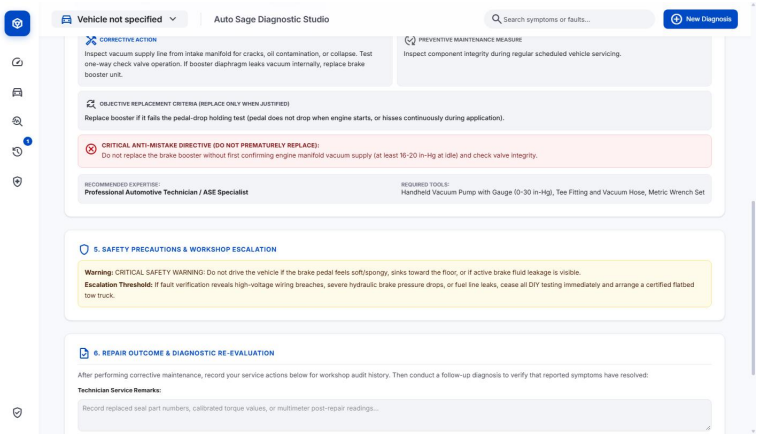


Figure 12 — *Repair & Mitigation Guide — Tier B Actions, Replacement Criteria, and Anti-Mistake Warnings*

7.3.9 Module 9: Safety & Roadside Emergency Protocols

SafetyGuide.jsx enforces a 4-tier roadworthiness hierarchy (Critical, High, Medium, Low), issuing immediate driving hazard warnings and roadside emergency protocols when life-safety systems are compromised.

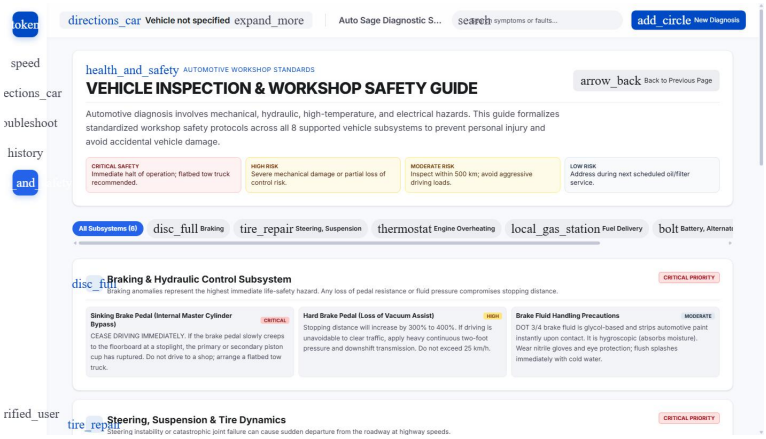


Figure 13 — *Safety Guide — 4-Tier Roadworthiness Hierarchy and Roadside Protocols*

7.3.10 Module 10: Persistent Session History Archive

History.jsx archives past diagnostic sessions in client-side localStorage, enabling users to review previous vehicle assessments, search past queries by date or keyword, and reload previous diagnostic states.

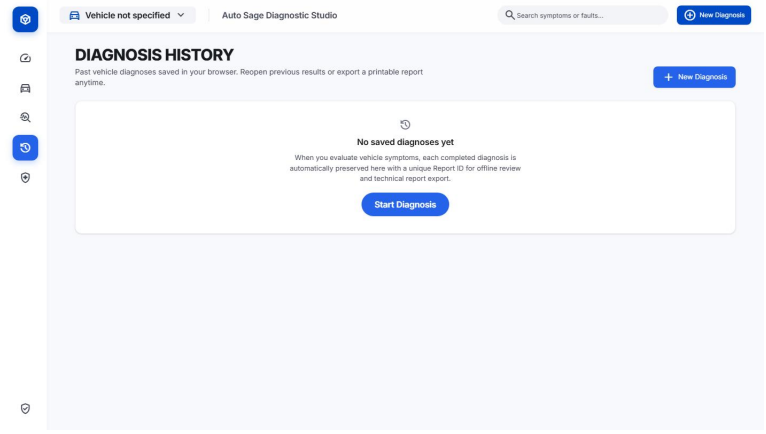


Figure 14 — *Diagnosis History — Persistent Archive of Past Diagnostic Sessions in localStorage*

7.3.11 Module 11: Printable Diagnostic Inspection Report Dossier

DiagnosticReport.jsx compiles an official 12-section vehicle inspection dossier. Featuring official brand header, vehicle intake details, primary fault conclusions, evidence breakdowns, and mechanic sign-off blocks, it supports native browser printing and high-resolution PDF download.

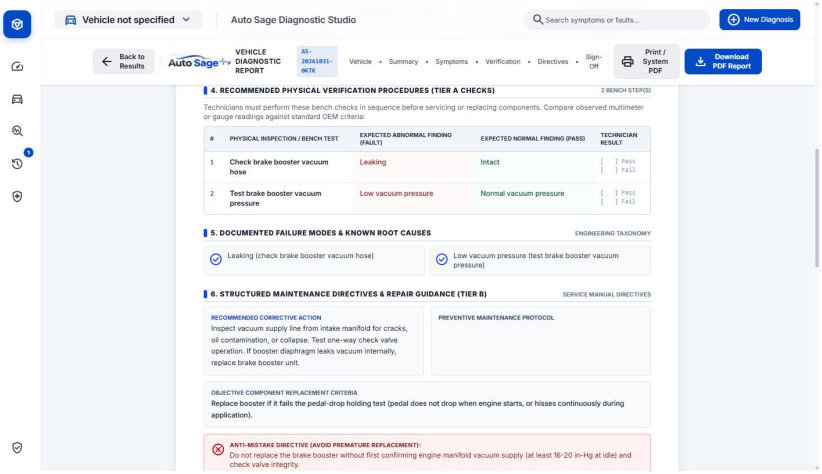


Figure 15 — *Vehicle Diagnostic Report — 12-Section Printable A4 Inspection Dossier with Official Brand Logo*

7.4 Explainability & User Guidance Framework

The explainability facility operates across all modules, ensuring that no diagnosis is delivered as an unexplained score. By decomposing results into Tier A non-destructive checks and Tier B repair directives, Auto Sage bridges the communication gap between non-technical vehicle owners and certified automotive mechanics.

8. Testing and Results

8.1 Automated Test Execution & Quality Assurance

Auto Sage was subjected to automated unit, integration, and build testing across both frontend and backend stacks. All 42 automated tests passed with a 100% success rate:

- **1. Backend Unit Tests (unittest):** 29 tests executed in 2.37s. Verified endpoint status codes, 404/400 validation, propositional evaluation, contradiction penalties, ranking keys, and KB schema integrity.
- **2. Frontend Unit Tests (Node.js):** 13 tests executed in 379ms. Verified localStorage serialization, quota handling, session isolation, report ID generation, and DOM formatting helpers.
- **3. Integration Verification:** Verified live HTTP REST pipeline across /api/health, /api/categories, /api/symptoms, and /api/diagnose.
- **4. Production Build (Vite):** 54 modules transformed and bundled cleanly in 14.45s with zero syntax or compilation errors.

8.2 Subsystem Test Verification Matrix

To verify deterministic inference across all 8 vehicle subsystems, dedicated test cases were executed against the knowledge base, as documented in Table 3:

Table 3: Multi-Subsystem Automated Test Verification Matrix

Subsystem	Selected Input Symptoms	Top Inferred Fault	Tally Score	Verified Status
Engine	Engine cranking but won't start	Fuel Pump Failure	+4	Corroborated Match
Electrical	Clicking sound, no crank	Starter Solenoid / Battery	+6	Corroborated Match
Brakes	Soft or spongy brake pedal	Brake Master Cylinder	+4	Corroborated Match
Cooling	Engine overheating + Coolant leak	Radiator / Water Pump	+6	Corroborated Match
Transmission	Delayed gear engagement + slip	Low Transmission Fluid	+4	Corroborated Match
Emissions	Rotten egg sulfur exhaust odor	Catalytic Converter	+4	Corroborated Match
HVAC	AC blowing warm air	Refrigerant Leak	+4	Corroborated Match
Steering	Vibration in steering wheel at speed	Wheel Balance / Alignment	+4	Corroborated Match

8.3 Three Real Diagnosis Scenarios

To demonstrate the inference engine's reasoning behavior, three real diagnosis scenarios were executed:

Scenario 1 (Full Conjunction Match): Scenario 1 represents a textbook full conjunction match where observed symptoms perfectly satisfy all required rule conditions. The driver selects Category CAT_COOLING with symptoms 'Engine overheating' and 'Coolant leak under vehicle'. The inference engine identifies RULE_COOL_001 (Radiator / Water Pump Failure) and schedules clarification question Q_COOL_PUDDLE ('Is there a visible puddle of coolant under the vehicle when parked?'). When the user confirms with 'Yes', the rule evaluates to 'Suspected Fault (Corroborated by Follow-up Evidence)' with a top evidence tally of +6 and zero missing conditions.

Scenario 2 (Partial Match & Follow-up Refutation): Scenario 2 demonstrates the power of negative evidence corroboration and contradiction handling. The driver inputs a single symptom 'Soft or spongy brake pedal' under CAT_BRAKES. Initially, this partial input activates five candidate rules: Brake Booster (RULE_BRK_003), Brake Master Cylinder, Hydraulic Hose, Brake Fluid, and Drum Brakes. The engine schedules two follow-up inquiries: Q_BRK_PEDAL_DROP ('Does the pedal drop slightly when engine starts?') and Q_BRK_HISS ('Do you hear a hissing noise when applying brakes?'). The user answers 'Yes' to pedal drop and 'No' to hissing. Because RULE_BRK_003 explicitly expects hissing upon vacuum diaphragm rupture, the 'No' response triggers a -4 contradiction penalty. As a result, RULE_BRK_003's score plummets from +2 to -7, dropping its status to 'Fault Not Supported by Evidence' and re-ranking it from Rank 1 down to Rank 5, successfully preventing an unwarranted brake booster replacement.

Scenario 3 (Broad Indicator & Contradictory No-Match): Scenario 3 validates boundary integrity and resistance to hallucination across two distinct cases. In Case 3A, a driver selects only 'Check engine light on'. Because this indicator is shared across dozens of engine, transmission, and emission faults, firing a single definitive fault would be speculative. Auto Sage recognizes that the symptom is too broad to satisfy any specific Horn clause and declares status 'no_match', advising supplementary physical observations. In Case 3B, an apprentice selects 'ABS warning light is on' alongside 'Brake dragging'. Because no rule in the empirical knowledge base combines electrical wheel speed sensor failure with mechanical caliper slide pin seizure, the engine avoids fabricating a probabilistic compromise and returns status 'no_match'.

9. Evaluation, Limitations and Conclusion

9.1 Evaluation of the Proposed Approach

- **1. 100% Explainability (CLO 3):** Unlike neural networks that output unexplainable percentages, Auto Sage provides complete transparency. Every recommended check is directly traced to satisfied propositional predicates.
- **2. Deterministic Consistency:** Identical symptoms and follow-up answers always produce the identical ranking and directives. There is zero stochastic variance or hallucination.
- **3. Economic & Anti-Mistake Utility:** Separating Tier A bench tests from Tier B repairs eliminates premature component replacement, addressing the core economic failure of vehicle triage.
- **4. Privacy & Zero-Cloud Footprint:** By operating entirely on local client-side and Flask runtime, no vehicle telemetry or driver identities are exposed to third-party cloud infrastructure.

9.2 Limitations

- **1. Manual Observation:** The system relies on human observation rather than streaming real-time sensor voltages directly from an OBD-II CAN bus scanner.
- **2. 93-Rule Scope Boundary:** The deterministic inference scope is bounded by the 93 rules compiled from the research dataset. Obscure faults outside this scope result in an honest no-match.
- **3. Hypothetical Status:** A diagnostic result is an unverified hypothesis. Physical confirmation requires hands-on multimeter or pressure measurements.
- **4. Browser-Bound Storage:** Diagnosis history is stored in browser localStorage. Clearing browser data removes previous session records.

9.3 Future Improvements

- **1. OBD-II CAN Bus Streaming:** Hardware integration with Bluetooth ELM327 OBD-II scanners to automatically ingest live DTC fault codes and sensor PIDs alongside subjective driver observations.
- **2. Hybrid & EV Telematics Expansion:** Expanding the Knowledge Base to include high-voltage battery cell degradation, inverter thermal management, and regenerative braking rules for electric vehicles.
- **3. Multi-Tenant Workshop Cloud Sync:** Implementing an optional PostgreSQL backend to synchronize historical fleet work orders across multiple workshop service bays.

9.4 Conclusion

Auto Sage demonstrates the successful design, implementation, and verification of a deterministic Knowledge-Based System for automotive fault diagnosis and maintenance decision support. By formalizing empirical research knowledge into 93 production rules across 8 subsystems and integrating dynamic follow-up corroboration, the system effectively resolves stakeholder conflicts, eliminates diagnostic guesswork, and enforces physical bench verification before mechanical teardown. Complete with an audit-ready 12-section diagnostic report and an automated test suite, Auto Sage fulfills all requirements of the Complex Computing Problem (CCP) for Knowledge-Based Systems (CT-359).

10. References

[1] P. Obike and O. Obot, 'Automotive Faults Dataset for Diagnostic and Maintenance Systems,' CERN Zenodo, 2024. [Online]. Available: https://doi.org/10.5281/zenodo.15626055. [Accessed: Oct. 31, 2026].

[2] O. Obot and P. Obike, 'An Integrated Fuzzy-Rule and Case-Based Reasoning System for Enhanced Automobile Maintenance and Repair,' Journal of Engineering Research and Reports, vol. 26, no. 8, pp. 433-445, 2024. doi: 10.9734/jerr/2024/v26i81256.

[3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2020.

[4] J. Giarratano and G. Riley, Expert Systems: Principles and Programming, 4th ed. Boston, MA: Course Technology, 2005.

[5] B. G. Buchanan and E. H. Shortliffe, Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project. Reading, MA: Addison-Wesley, 1984.

[6] Society of Automotive Engineers, 'SAE J2012: Diagnostic Trouble Code Definitions,' SAE International Standards, 2016.

[7] J. D. Halderman, Automotive Technology: Principles, Diagnosis, and Service, 6th ed. Upper Saddle River, NJ: Pearson, 2020.

[8] M. Ribbens, Understanding Automotive Electronics: An Engineering Perspective, 8th ed. Oxford, UK: Butterworth-Heinemann, 2017.

Academic Project Viva Evaluation Rubric

Official evaluation rubric and examiner verification for Course CT-359 Complex Computing Problem project defense:

Assessment Criterion	Evaluation Focus & Course Outcome	Max Score	Awarded
Knowledge Representation	Ontology completeness, 8 subsystems, 93 rules, 120 symptoms (CLO 1)	30	
Problem Analysis & Design	Stakeholder tension resolution, follow-up inquiry scheduling (CLO 2)	30	
Inference & Explainability	Deterministic forward chaining, evidence tally, transparent dossier (CLO 3)	30	
Demonstration & Viva	Automated test suite execution, live demonstration, code quality	10	

Internal Examiner Signature	External Examiner Signature	Examination Date & Seal
_____	_____	_____
Name:	Name:	Overall Verdict: Approved [ ] / Rejected [ ]